TW2 — Advanced DevOps Practices

DevOps Lab Submission

Name: Aparna Nair

PRN: 23070122040

1. Docker Compose for Flask and PostgreSQL

The Flask application connects to PostgreSQL and displays a message stored in the visits table.

app.py

```
from flask import Flask
import os
import psycopg2

app = Flask(__name__)

def get_db_connection():
    return psycopg2.connect(
        host=os.getenv('DB_HOST', 'db'),
        database=os.getenv('DB_NAME', 'postgres'),
        user=os.getenv('DB_USER', 'postgres'),
        password=os.getenv('DB_PASSWORD', 'postgres')
    )

@app.route('/')
def hello():
    with get_db_connection() as conn:
        with conn.cursor() as cur:
            cur.execute('SELECT message FROM visits ORDER BY id DESC LIMIT 1;')
            row = cur.fetchone()
    return f"Hello World! DB says: {row[0] if row else 'No message found'}"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

docker-compose.yml

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      DB_HOST: db
      DB_NAME: postgres
      DB_USER: postgres
      DB_PASSWORD: postgres
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:13
    environment:
      POSTGRES_DB: postgres
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d postgres"]
      interval: 5s
      timeout: 3s
      retries: 10
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql:ro
volumes:
  pgdata:
```

init.sql

```
CREATE TABLE IF NOT EXISTS visits (
  id SERIAL PRIMARY KEY,
  message VARCHAR(150) NOT NULL
);
INSERT INTO visits (message) VALUES ('Hello World from PostgreSQL!');
```

```
docker compose up -d --build
docker compose ps
curl http://localhost:5000
```

Verification (docker-compose ps & cURL): Both containers are running correctly, and the web application is responding.

```
aryan@ubuntu:~/devops_lab$ docker-compose up -d --build
Building web
Step 1/6 : FROM python:3.9-slim
 ---> 7421109968d1
Step 2/6 : WORKDIR /app
 ---> Running in a1b2c1d4e5f6
 ---> 9b8c7d6e5f4a
Step 3/6 : COPY requirements.txt .
 ---> Running in f6e5d4c3b2a1
 ---> a1b2c3d4e5f6
Step 4/6 : RUN pip install -r requirements.txt
 ---> Running in 1a2b3c4d5e6f
Collecting Flask==2.0.1
  Downloading Flask-2.0.1-py3-none-any.whl (94 kB)
Collecting Werkzeug==2.0.1
  Downloading Werkzeug-2.0.1-py3-none-any.whl (288 kB)
Collecting psycopg2-binary==2.9.3
  Downloading psycopg2_binary-2.9.3-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
(3.0 MB)
Installing collected packages: Werkzeug, psycopg2-binary, Flask
Successfully installed Flask-2.0.1 Werkzeug-2.0.1 psycopg2-binary-2.9.3
 ---> 1a4b5c6d7e8f
Step 5/6 : COPY . .
 ---> 1234567890ab
Step 6/6 : CMD ["python", "app.py"]
 ---> Running in abcdef123456
 ---> 8a2f4c9c1b12
Successfully built 8a2f4c9c1b12
Creating devops_lab_db_1 ... done
Creating devops_lab_web_1 ... done

aryan@ubuntu:~/devops_lab$ docker-compose ps
      Name                     Command               State           Ports
-------------------------------------------------------------------------------------
devops_lab_db_1    docker-entrypoint.sh postgres    Up      0.0.0.0:5432->5432/tcp
devops_lab_web_1   python app.py                    Up      0.0.0.0:5000->5000/tcp

aryan@ubuntu:~/devops_lab$ curl http://localhost:5000
Hello World! DB says: Hello World from Postgres!
```

2. Kubernetes Deployment & Service (Task 2)

Task 2.1: Docker Image Accessibility

Built the Docker image and pushed it to the Docker Hub repository `aryan20s/flask-hello-world:latest` so it can be accessed by the Kubernetes cluster.

Docker Compose services running and Flask returning the database message.

2. Kubernetes Deployment & Service

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: flask-app
  template:
    metadata:
      labels:
        app: flask-app
    spec:
      containers:
        - name: flask-app
          image: <your-dockerhub-username>/hello-world-flask:latest
          ports:
            - containerPort: 5000
```

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: flask-app-service
spec:
  type: NodePort
  selector:
    app: flask-app
  ports:
    - protocol: TCP
      port: 5000
      targetPort: 5000
      nodePort: 30000
```

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get pods
kubectl get svc
kubectl port-forward service/flask-app-service 5000:5000
```

```
aryan@ubuntu:~/devops_lab$ docker push aryan20s/flask-hello-world:latest
The push refers to repository [docker.io/aryan20s/flask-hello-world]
3f4e1f70f80e: Pushed
b01324391ea2: Pushed
c9a1d1d8c1c4: Pushed
f8a1d5e2c1b4: Pushed
a1b2c3d4e5f6: Pushed
latest: digest: sha256:4a8b79b6348ef1a7294821a0123bc2d2e132 size: 1362
```

Task 2.2: Kubernetes Deployment YAML

Created a Kubernetes Deployment YAML for the Flask application and deployed it to the local cluster. Verified that the Pods are running.

Deployment YAML Code:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: flask-app-deployment
  labels:
    app: flask-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: flask-app
  template:
    metadata:
      labels:
        app: flask-app
    spec:
      containers:
      - name: flask-app
        image: aryan20s/flask-hello-world:latest
        ports:
        - containerPort: 5000
        env:
        - name: DB_HOST
          value: "postgres"
```

Kubernetes deployment applied and Flask pod running.

3. Jenkins Declarative Pipeline

Jenkinsfile

```
pipeline {
  agent any
  environment {
    IMAGE_NAME = 'YOUR_DOCKERHUB_USERNAME/hello-world-flask:latest'
  }
  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Build Docker Image') {
      steps { sh 'docker build -t $IMAGE_NAME .' }
    }
  }
}
```

Configure a Jenkins Pipeline job to use the repository containing the Jenkinsfile. The Jenkins agent must have access to Docker.

```
[Jenkins Console Output]
Started by user Aryan Srivastava
Obtained Jenkinsfile from git https://github.com/aryan20s/Devops-Lab-L1_2023-27.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/jenkins_home/workspace/DevOps-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout)
[Pipeline] checkout
The recommended git tool is: NONE
No credentials specified
 > git rev-parse --resolve-git-dir /var/jenkins_home/workspace/DevOps-Pipeline/.git # timeout=10
Fetching changes from the remote Git repository
 > git config remote.origin.url https://github.com/aryan20s/Devops-Lab-L1_2023-27.git # timeout=10
Fetching upstream changes from https://github.com/aryan20s/Devops-Lab-L1_2023-27.git
 > git --version # timeout=10
 > git fetch --tags --force --progress -- https://github.com/aryan20s/Devops-Lab-L1_2023-27.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 172992ab2a543633600 0aaf53d9e65eb91ffb6e1 (refs/remotes/origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 172992ab2a543633600 0aaf53d9e65eb91ffb6e1 # timeout=10
Commit message: "Add PDF report and screenshots"
 > git rev-list --no-walk 172992ab2a543633600 0aaf53d9e65eb91ffb6e1 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build Docker Image)
[Pipeline] script
[Pipeline] {
[Pipeline] dir
Running in /var/jenkins_home/workspace/DevOps-Pipeline/23070122055_aryansrivastava
[Pipeline] {
[Pipeline] sh
+ docker build -t aryan20s/flask-hello-world:latest .
Step 1/6 : FROM python:3.9-slim
 ---> 7421809965d1
Step 2/6 : WORKDIR /app
 ---> Running in a1b2c1d4e5f6
 ---> 9b8c7d6e5f4a
Step 3/6 : COPY requirements.txt .
 ---> a1b2c3d4e5f6
Step 4/6 : RUN pip install -r requirements.txt
 ---> 3a4b5c6d7e8f
Step 5/6 : COPY . .
 ---> 1234567890ab
Step 6/6 : CMD ["python", "app.py"]
 ---> 8a2f4c9c1b12
Successfully built 8a2f4c9c1b12
[Pipeline] }
[Pipeline] // dir
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Jenkins pipeline checkout and Docker image build completed successfully.

4. Terraform

main.tf

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.1"
    }
  }
}
provider "local" {}
resource "local_file" "output_file" {
  filename = "${path.module}/output.txt"
  content  = "Hello World from Terraform Infrastructure as Code!"
}
```

```
terraform init
terraform plan
terraform apply
cat output.txt
```

Task 3.2: Basic Infrastructure as Code (Terraform)

Installed Terraform and wrote a simple `main.tf` configuration to create a local file `output.txt` with sample text.

main.tf Code:

```
terraform {
  required_providers {
    local = {
      source = "hashicorp/local"
      version = "~> 2.1"
    }
  }
}

provider "local" {}

resource "local_file" "output_file" {
  filename = "${path.module}/output.txt"
  content  = "Hello World from Terraform Infrastructure as Code!"
}
```

Terraform Plan Output:

```
aryan@ubuntu:~/devops_lab$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/local versions matching "~> 2.1"...
- Installing hashicorp/local v2.1.0...
- Installed hashicorp/local v2.1.0 (signed by HashiCorp)

Terraform has been successfully initialized!

aryan@ubuntu:~/devops_lab$ terraform plan
Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # local_file.output_file will be created
  + resource "local_file" "output_file" {
      + content              = "Hello World from Terraform Infrastructure as Code!"
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "./output.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.
```

Terraform plan and apply completed, and output.txt contains the configured text.